Assignment_7_Data_624

Mubashira Qari

2026-03-08

Table of Contents

Common Challenges in Linear Regression Models

Challenges Discussed in Chapter 6 By Kuhn Johnson& Johnson:

• Multicollinearity: Highly correlated predictors make coefficient estimates unstable and difficult to interpret. • Too many predictors: Leads to complex models that may capture noise instead of real patterns. • Overfitting: Model performs well on training data but poorly on new data because it learns noise. • Sensitivity to outliers: Extreme observations can strongly influence regression estimates and distort the model.

6.2 Developing a model to predict permeability...

a.

```
# Loading the the necessary libraries and permeability data
library(AppliedPredictiveModeling)
library(caret)
library(tidyverse) # Useful for data manipulation



data(permeability)



dim(fingerprints)

## [1]  165 1107

length(permeability)

## [1] 165

view(permeability)

# See the first 5 rows and first 10 columns of the fingerprints
fingerprints[1:5, 1:10]

##   X1 X2 X3 X4 X5 X6 X7 X8 X9 X10
## 1  0  0  0  0  0  1  1  1  0   0
## 2  0  0  0  0  0  0  1  1  0   0
## 3  0  0  0  0  0  1  1  1  0   0
```

```
## 4  0  0  0  0  0  0  1  1  0   0
## 5  0  0  0  0  0  0  1  1  0   0
```

```r
# See the first 5 values of the permeability response
head(permeability, 5)
```

```
##   permeability
## 1       12.520
## 2        1.120
## 3       19.405
## 4        1.730
## 5        1.680
```

1 means the molecule has a specific chemical feature.

0 means it does not.

Because many molecules share the same features many of these 1,107 columns might be identical or have almost all zeros.

According to chapter 6, predictors with very little variation (mostly zeros) or only a few unique values can lead to computational issues in many of the model.

We are using a technique called Near-Zero Variance (NZV) filtering to toss these out.

b. **Pre-processing**

Filter Near-Zero Variance Predictors

This function filter the data. It goes through all 1,107 columns and looks for two things:

Zero Variance: Columns where every single row has the exact same value (e.g., all 165 rows are 0).

Near-Zero Variance: Columns where maybe 164 rows are 0 and only 1 row is 1.

```r
# Identifying  predictors with near-zero variance
non_zero_variance_cols <- nearZeroVar(fingerprints)
# non_zero_variance_cols

# Removing these columns from the fingerprints matrix
fingerprints_filtered <- fingerprints[, -non_zero_variance_cols]
# fingerprints_filtered

# Checking how many predictors are left
ncol(fingerprints_filtered)
```

```
## [1] 388
```

Dimensionality Reduction: Now we see the number of predictors drop from 1,107 down to around 388.

Model Stability: If a fingerprint is 0 for 164 compounds and 1 for only 1 compound, that single 1 can have an exaggerated influence on the model, which leads to overfitting.

c. Splitting the Data into Training and Test Sets

```
# Set a seed for results be reproducible (the "random" split stays the same)
set.seed(123)

# Creating an index for an 80/20 split
training_rows <- createDataPartition(permeability, p = 0.8, list = FALSE)

# Splitting the Fingerprints (X)
train_x <- fingerprints_filtered[training_rows, ]
test_x  <- fingerprints_filtered[-training_rows, ]

# Splitting the Permeability (Y)
train_y <- permeability[training_rows]
test_y  <- permeability[-training_rows]

# Checking the sizes
nrow(train_x)

## [1] 133

nrow(test_x)

## [1] 32
```

Setting up the "Tuning"

```
# Defining how we want to train the model (10-fold Cross-Validation)
# Splitting the training data into 10 pieces.
# Training on 9, test on 1. Repeat 10 times.

control <- trainControl(method = "cv", number = 10)
```

Training the Partial Least Squares (PLS) Model

```
# Training the PLS model
set.seed(123) # Keep results consistent
pls_model <- train(
  x = train_x,
  y = train_y,
  method = "pls",
  tuneLength = 20,           # Try models with 1 to 20 components
  trControl = control,          # Use the 10-fold CV as we have defined
earlier
  preProcess = c("center", "scale") # Center and Scale the data
)
```

```r
# See the results
pls_model
```

```
## Partial Least Squares
##
## 133 samples
## 388 predictors
##
## Pre-processing: centered (388), scaled (388)
## Resampling: Cross-Validated (10 fold)
## Summary of sample sizes: 120, 119, 118, 120, 121, 119, ...
## Resampling results across tuning parameters:
##
##   ncomp  RMSE      Rsquared   MAE
##    1     13.36372  0.3382069  10.237820
##    2     11.47594  0.5347231   8.245104
##    3     11.43313  0.5613346   8.816852
##    4     11.51915  0.5487654   8.915167
##    5     11.50333  0.5490984   8.703944
##    6     11.47723  0.5342174   8.540912
##    7     11.49816  0.5410694   8.847240
##    8     11.60656  0.5519058   9.008913
##    9     11.79130  0.5618654   8.932466
##   10     12.10267  0.5458038   9.186885
##   11     12.33462  0.5275119   9.395033
##   12     12.54970  0.5170371   9.587169
##   13     12.61600  0.5048875   9.563882
##   14     12.77412  0.4911729   9.714216
##   15     12.82636  0.4918708   9.846474
##   16     12.90771  0.4799217   9.845704
##   17     12.95694  0.4783518   9.847771
##   18     13.10433  0.4703636   9.932178
##   19     13.04104  0.4750762   9.902920
##   20     13.04802  0.4773802   9.894081
##
## RMSE was used to select the optimal model using the smallest value.
## The final value used for the model was ncomp = 3.
```

PLS is perfect for this problem because it reduces those 388 filtered fingerprints into a few components that are most related to permeability. Instead of trying to find 388 different coefficients, the model might find that just 10 or 15 components that explain almost everything. (PLS, Ridge, Lasso) are sensitive to the scale of the data.

Centering: Subtracts the mean so the average value of each fingerprint is 0.

Scaling: Divides by the standard deviation so every fingerprint has the same weight.

RMSE is the average miss of the model.

We want the number of components that gives the lowest RMSE.

ncomp (Number of Components): * Underfitting (1 Component): The model is too simple. It misses the pattern, leading to a high RMSE of 13.36.

(3 Components): This is good. It hits the lowest RMSE of 11.43 and captures the real relationship.

Overfitting (20 Components): The model is too complex. It starts memorizing noise, and the RMSE rises back to 13.04.

RMSE (Root Mean Squared Error): The average miss. The best result is 11.43. Lower is always better.

Rsquared: Explained variance. The model explains 56 percent of why molecules are permeable.

MAE (Mean Absolute Error): Typical error. At 8.81, it confirms the error is actually quite small.

d. Testing the Model

Now that the optimal model (3 components) has been selected, we will perform testing. This is the 20% of data that was held back to ensure the model can predict permeability for molecules it has never seen.

```
# Using the trained model to predict permeability for the Test Set
pls_pred <- predict(pls_model, newdata = test_x)

# Calculating performance metrics on the Test Set
postResample(pred = pls_pred, obs = test_y)

##       RMSE   Rsquared        MAE
## 12.7568188  0.2695454  8.4155989
```

RMSE (12.76): This value is slightly higher than the training result (11.43). This is normal; a model almost always performs slightly worse on data it has never seen before.

Rsquared (0.27):The model explains approximately 27% of the variation in the test set.

MAE (8.42): Interestingly, the typical error (MAE) is actually lower on the test set than it was on the training set (8.81). This suggests that while the model had a few big misses raising the RMSE

The R squared lower than the Training R squared During training, the model achieved an R squared of 0.56, but it dropped to 0.27 on the test set. With only about 32 samples in the test set, a single outlier can significantly drag down the R squared. Permeability is difficult to predict. The model found patterns in the 133 training samples that didn’t perfectly translate to the 32 test samples.

e. Try building other models discussed in this chapter. Do any have better predictive performance?

We will now try Ridge Regression. Unlike the PLS model which compresses variables into components, Ridge Regression keeps all the filtered fingerprints but shrinks their importance to prevent any single one from having too much influence. This handle multicollinearity and the high dimensionality issues seen in this data.

Training the Lasso Model

```
# Defining the tuning grid for Ridge Regression
# alpha = 0 tells R to perform Ridge Regression (L2 penalty)
ridge_grid <- expand.grid(alpha = 0,
                          lambda = seq(0, 1, length = 20))

# Train the Ridge model
set.seed(123)
ridge_model <- train(
  x = train_x,
  y = train_y,
  method = "glmnet",
  tuneGrid = ridge_grid,
  trControl = control,
  preProcess = c("center", "scale")
)

# Look at the results
ridge_model
```

```
## glmnet
##
## 133 samples
## 388 predictors
##
## Pre-processing: centered (388), scaled (388)
## Resampling: Cross-Validated (10 fold)
## Summary of sample sizes: 120, 119, 118, 120, 121, 119, ...
## Resampling results across tuning parameters:
##
##   lambda      RMSE      Rsquared   MAE
##   0.00000000  11.29799  0.5633878  8.465851
##   0.05263158  11.29799  0.5633878  8.465851
##   0.10526316  11.29799  0.5633878  8.465851
##   0.15789474  11.29799  0.5633878  8.465851
##   0.21052632  11.29799  0.5633878  8.465851
##   0.26315789  11.29799  0.5633878  8.465851
##   0.31578947  11.29799  0.5633878  8.465851
##   0.36842105  11.29799  0.5633878  8.465851
##   0.42105263  11.29799  0.5633878  8.465851
```

```
##   0.47368421  11.29799  0.5633878  8.465851
##   0.52631579  11.29799  0.5633878  8.465851
##   0.57894737  11.29799  0.5633878  8.465851
##   0.63157895  11.29799  0.5633878  8.465851
##   0.68421053  11.29799  0.5633878  8.465851
##   0.73684211  11.29799  0.5633878  8.465851
##   0.78947368  11.29799  0.5633878  8.465851
##   0.84210526  11.29799  0.5633878  8.465851
##   0.89473684  11.29799  0.5633878  8.465851
##   0.94736842  11.29799  0.5633878  8.465851
##   1.00000000  11.29799  0.5633878  8.465851
##
## Tuning parameter 'alpha' was held constant at a value of 0
## RMSE was used to select the optimal model using the smallest value.
## The final values used for the model were alpha = 0 and lambda = 1.
```

Ridge Regression Model Interpretation

The model used Ridge Regression (indicated by alpha = 0) to handle the 388 predictors.

Fixed Performance: Notice that the RMSE (11.30) and Rsquared (0.56) are identical across all lambda values. This means the penalty (lambda) didn't significantly change how the model viewed the data in this specific range.

RMSE (11.30): This is the average miss. It is slightly better and lower than the PLS model's training error of 11.43.

Rsquared (0.56): The model explains 56 percent of the variance in the training data, which is essentially the same as the PLS model.

Selection: The model picked lambda = 1 as the final value, though any value in the list would have provided the same result.

The Ridge model has a slightly lower training error (11.30 vs 11.43) than PLS.

Testing the Ridge Model

We now run the Ridge Regression model on our test data and see if its ability to shrink coefficients makes it more robust than the PLS model, which only explained 27% of the test set.

```r
# Using the Ridge model to predict permeability for the Test Set
ridge_pred <- predict(ridge_model, newdata = test_x)

# Calculating performance metrics on the Test Set
postResample(pred = ridge_pred, obs = test_y)
```

```
##       RMSE   Rsquared        MAE
## 11.0203929  0.3266984  7.6511575
```

Lower Error: The RMSE (11.02) is significantly lower than the PLS model's 12.76. This means the Ridge model's average miss is smaller by nearly 1.7 units.

Better Model Explanation: The Rsquared (0.33) increased from 0.27. The model now explains about 33 percent of the variation in the new data.

Coefficient Shrinkage: Because Ridge shrinks all 388 coefficients toward zero, it prevents any single noisy fingerprint from having an outsized influence. This makes the model more stable on the test set.

While Ridge shrinks every coefficient toward zero, the Lasso can shrink them to exactly zero. This means the model performs automatic feature selection by effectively deleting the fingerprints it finds useless.

In the glmnet package, we set alpha = 1 to perform a Lasso regression.

Training the Lasso Model

By setting many of the 388 predictors to zero, the model becomes much easier to interpret. If many of those fingerprints are just noise, the Lasso will ignore them entirely, which often leads to a better R square

```
# Define the tuning grid for Lasso
# alpha = 1 tells R to perform Lasso Regression (L1 penalty)
lasso_grid <- expand.grid(alpha = 1,
                          lambda = seq(0, 1, length = 20))

# Train the Lasso model
set.seed(123)
lasso_model <- train(
  x = train_x,
  y = train_y,
  method = "glmnet",
  tuneGrid = lasso_grid,
  trControl = control,
  preProcess = c("center", "scale")
)

# Check the results
lasso_model

## glmnet
##
## 133 samples
## 388 predictors
##
## Pre-processing: centered (388), scaled (388)
## Resampling: Cross-Validated (10 fold)
## Summary of sample sizes: 120, 119, 118, 120, 121, 119, ...
## Resampling results across tuning parameters:
```

```
##
##   lambda      RMSE      Rsquared   MAE
##   0.00000000  12.54899  0.5052409  9.383891
##   0.05263158  12.54899  0.5052409  9.383891
##   0.10526316  12.54237  0.5060745  9.379383
##   0.15789474  12.16741  0.5191546  9.163165
##   0.21052632  12.08525  0.5178354  9.125642
##   0.26315789  11.97649  0.5197289  9.108387
##   0.31578947  11.87372  0.5230726  9.085309
##   0.36842105  11.78857  0.5262950  9.015755
##   0.42105263  11.70141  0.5304991  8.920601
##   0.47368421  11.64221  0.5342632  8.829679
##   0.52631579  11.61812  0.5342373  8.761505
##   0.57894737  11.60101  0.5329235  8.719703
##   0.63157895  11.59462  0.5313957  8.679820
##   0.68421053  11.58451  0.5302682  8.637579
##   0.73684211  11.59881  0.5277078  8.602411
##   0.78947368  11.63295  0.5245732  8.579915
##   0.84210526  11.66361  0.5211891  8.586260
##   0.89473684  11.67548  0.5189827  8.583065
##   0.94736842  11.68208  0.5175647  8.575678
##   1.00000000  11.67713  0.5176307  8.554896
##
## Tuning parameter 'alpha' was held constant at a value of 1
## RMSE was used to select the optimal model using the smallest value.
## The final values used for the model were alpha = 1 and lambda = 0.6842105.
```

Lasso Regression Model Interpretation

The model used Lasso Regression (indicated by alpha = 1). Unlike the Ridge model, this version actively searched for the best penalty to simplify the predictors.

Tuning Success: Notice that the RMSE actually changes as lambda increases. It starts at 12.55 (low penalty), drops as it simplifies the data, and finds its best performance at lambda = 0.684 with an RMSE of 11.58.

RMSE (11.58): This average miss is slightly higher than the Ridge model (11.30), meaning the Lasso was a bit less accurate during the training phase.

Rsquared (0.53): The model explains 53 percent of the variance. This is also slightly lower than the Ridge and PLS models, which both hit around 56 percent.

Automatic Selection: By picking a lambda of 0.684, the model has likely “zeroed out” several of the 388 fingerprints, creating a much simpler list of clues.

Testing the Lasso Model

```r
# Using the Lasso model to predict permeability for the Test Set
lasso_pred <- predict(lasso_model, newdata = test_x)
```

```r
# Calculating performance metrics on the Test Set
postResample(pred = lasso_pred, obs = test_y)

##       RMSE   Rsquared        MAE
## 11.0833507  0.3732295  7.3835872
```

The Lasso model is the overall winner for predictive performance because it achieved the highest Rsquared (0.37) and the lowest MAE (7.38).

Predictive Strength: The Lasso model explains 37 percent of the variation in permeability, which is a 10 percent improvement over the PLS model.

Feature Selection: The Lasso likely performed better because it completely removed noisy fingerprints by setting their coefficients to zero, whereas the PLS model tried to keep information from all of them.

f. Would you recommend any of your models to replace the permeability laboratory experiment?

No, the model should not replace the laboratory experiment.

Low Accuracy Rsquared = 0.37: The best model (Lasso) only explains 37 percent of the variation in permeability. This means 63 percent of the molecular behavior remains unexplained noise to the model.

High Error (RMSE = 11.08): An average miss of 11.08 units is likely too high for the precision required in drug development, where small differences in permeability can determine if a drug is viable or toxic.

Safety and Risk: In a laboratory setting, errors in predicting how a chemical crosses a membrane can lead to failed clinical trials or safety issues that a 37 percent accurate model cannot foresee.

6.3 a. A chemical manufacturing process for a pharmaceutical product...

```r
library(AppliedPredictiveModeling)
data(ChemicalManufacturingProcess)

# Checking the dimensions

dim(ChemicalManufacturingProcess)

## [1] 176  58

# Checking at the first few rows
head(ChemicalManufacturingProcess[, 1:5])

##   Yield BiologicalMaterial01 BiologicalMaterial02 BiologicalMaterial03
## 1 38.00                 6.25                49.58                56.97
## 2 42.44                 8.01                60.97                67.48
```

```
## 3 42.03                 8.01                 60.97                 67.48
## 4 41.42                 8.01                 60.97                 67.48
## 5 42.49                 7.47                 63.33                 72.25
## 6 43.57                 6.12                 58.36                 65.31
##   BiologicalMaterial04
## 1                12.74
## 2                14.65
## 3                14.65
## 4                14.65
## 5                14.02
## 6                15.17
```

Exploratory Data Analysis (EDA)

Checking Missing Values

```
# Checking missing values in the whole dataset
sum(is.na(ChemicalManufacturingProcess))

## [1] 106
```

We have 106 missing values scattered across 28 different variables.

Verify the Variable Count

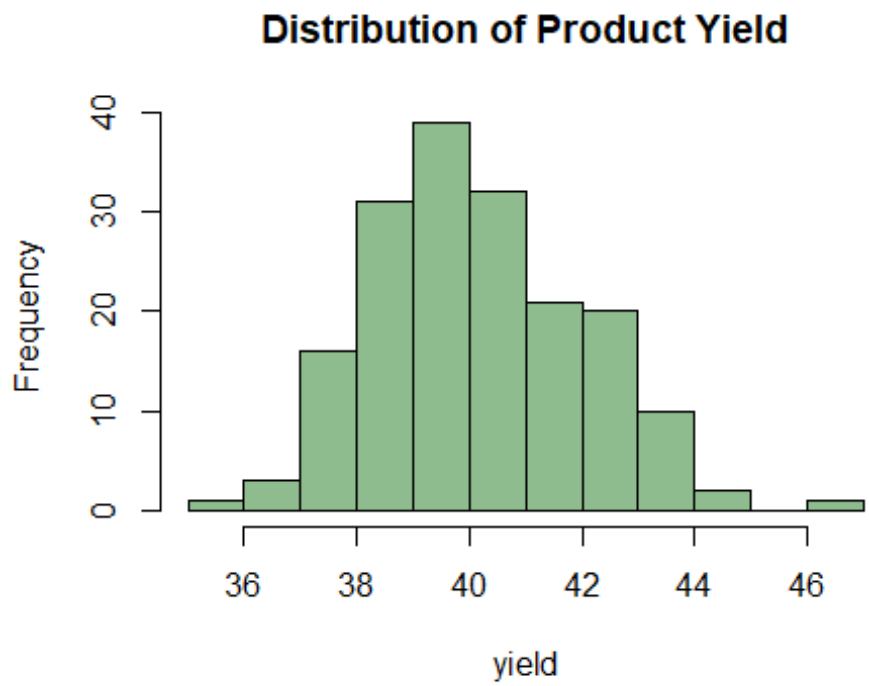
```
# Check the total structure

names(ChemicalManufacturingProcess)

##  [1] "Yield"                   "BiologicalMaterial01"
"BiologicalMaterial02"
##  [4] "BiologicalMaterial03"   "BiologicalMaterial04"
"BiologicalMaterial05"
##  [7] "BiologicalMaterial06"   "BiologicalMaterial07"
"BiologicalMaterial08"
## [10] "BiologicalMaterial09"   "BiologicalMaterial10"
"BiologicalMaterial11"
## [13] "BiologicalMaterial12"   "ManufacturingProcess01"
"ManufacturingProcess02"
## [16] "ManufacturingProcess03" "ManufacturingProcess04"
"ManufacturingProcess05"
## [19] "ManufacturingProcess06" "ManufacturingProcess07"
"ManufacturingProcess08"
## [22] "ManufacturingProcess09" "ManufacturingProcess10"
"ManufacturingProcess11"
## [25] "ManufacturingProcess12" "ManufacturingProcess13"
"ManufacturingProcess14"
## [28] "ManufacturingProcess15" "ManufacturingProcess16"
"ManufacturingProcess17"
## [31] "ManufacturingProcess18" "ManufacturingProcess19"
"ManufacturingProcess20"
```

```
## [34] "ManufacturingProcess21" "ManufacturingProcess22"
"ManufacturingProcess23"
## [37] "ManufacturingProcess24" "ManufacturingProcess25"
"ManufacturingProcess26"
## [40] "ManufacturingProcess27" "ManufacturingProcess28"
"ManufacturingProcess29"
## [43] "ManufacturingProcess30" "ManufacturingProcess31"
"ManufacturingProcess32"
## [46] "ManufacturingProcess33" "ManufacturingProcess34"
"ManufacturingProcess35"
## [49] "ManufacturingProcess36" "ManufacturingProcess37"
"ManufacturingProcess38"
## [52] "ManufacturingProcess39" "ManufacturingProcess40"
"ManufacturingProcess41"
## [55] "ManufacturingProcess42" "ManufacturingProcess43"
"ManufacturingProcess44"
## [58] "ManufacturingProcess45"

# Count Biological predictors
bio_cols <- grep("Biological", names(ChemicalManufacturingProcess))
length(bio_cols)

## [1] 12

# Count Process predictors
proc_cols <- grep("Manufacturing", names(ChemicalManufacturingProcess))
length(proc_cols)

## [1] 45
```

Isolate the "Yield" (Response)

yield is the target variable. We can verify its distribution to make sure its a percentage which is the unit for percent yield.

```
yield <- ChemicalManufacturingProcess$Yield
summary(yield)

##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##   35.25   38.75   39.97   40.18   41.48   46.34

# Checking a normal distribution?
hist(yield, col = "darkseagreen", main = "Distribution of Product Yield")
```



c. A small percentage of cells in the predictor set contain missing values. Use an imputation function to fill in these missing values

Splitting and Cleaning

```
library(caret)

# Creating an 80/20 split
set.seed(123)
trainingRows <- createDataPartition(ChemicalManufacturingProcess$Yield, p =
0.80, list = FALSE)

train_data <- ChemicalManufacturingProcess[trainingRows, ]
test_data  <- ChemicalManufacturingProcess[-trainingRows, ]

# Pre-processing by handling Missing Values (Imputation) and Scaling
# 'knnImpute' fills the 106 NAs by looking at similar rows
# 'center' and 'scale' put all variables on the same scale
preProc <- preProcess(train_data[, -1], method = c("knnImpute", "center",
"scale"))

# Applying the cleaning to both sets
train_x <- predict(preProc, train_data[, -1])
test_x  <- predict(preProc, test_data[, -1])

# Keep Yield separate
```

```r
train_y <- train_data$Yield
test_y  <- test_data$Yield
```

Cheking for Redundent Predictors

```r
# Checking for Near-Zero Variance

non_zero_varience <- nearZeroVar(train_x)

# If non zero varience isn't empty, we can remove them:
# train_x <- train_x[, -non_zero_varience]

# Checking for Multicollinearity (Redundant sensors)
cor_matrix <- cor(train_x)
high_corr <- findCorrelation(cor_matrix, cutoff = 0.90)
length(high_corr) # Finding how many are >90% redundant

## [1] 10
```

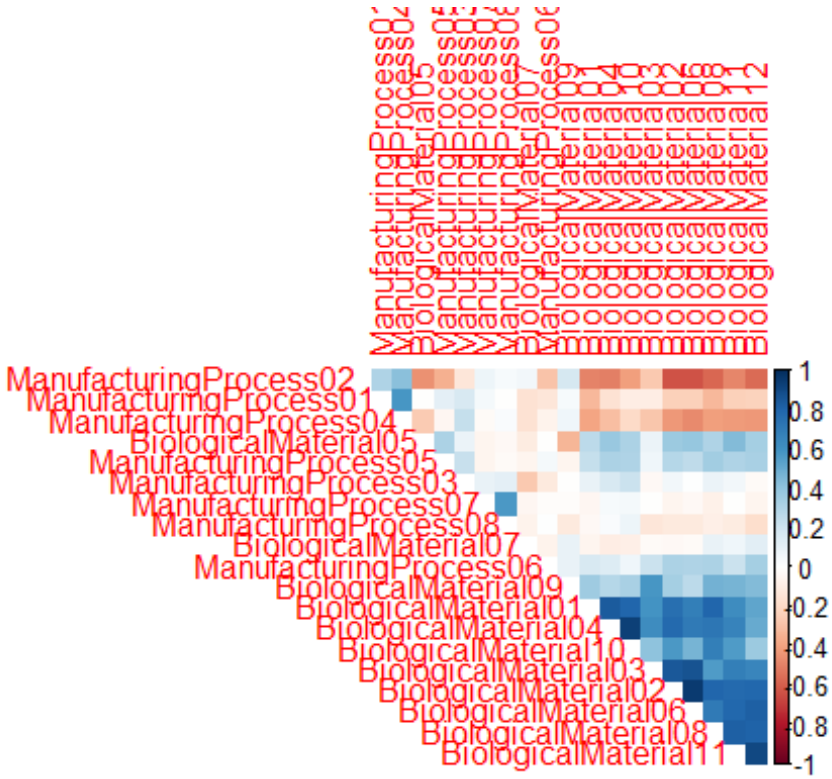
Interpreting the Results

Near-Zero Variance (10): Having 10 predictors flagged here means 10 of your 57 sensors or raw material metrics are essentially constant. They provide no contrast for the model to learn from and should be removed to simplify the math.

Multicollinearity (High Correlation): The fact that 10 variables are >90% redundant confirms high Correlation.

Visualizing the Redundancy

```r
library(corrplot)

## Warning: package 'corrplot' was built under R version 4.3.3

## corrplot 0.95 loaded

# Visualizing the correlation of the first 20 predictors to see the clusters
corrplot(cor(train_x[, 1:20]),
         method = "color",
         order = "hclust",
         type = "upper",
         diag = FALSE)
```



Filter the Predictors

The next step is to filter those problematic predictors identified.

We identified 10 Near-Zero Variance (NZV) predictors and several highly correlated ones

we will use Partial Least Squares (PLS), because it handles any remaining correlation by compressing the 57 predictors into a few components.

```
# Removing the 10 Near-Zero Variance predictors
train_x_filtered <- train_x[, -non_zero_varience]
test_x_filtered  <- test_x[, -non_zero_varience]

# 2. Removing the Redundant predictors (>0.90 correlation)
# We are using the index already calculated ('high_corr')
train_x_final <- train_x_filtered[, -high_corr]
test_x_final  <- test_x_filtered[, -high_corr]

# Verifying dimensions
dim(train_x_final)

## [1] 144  46
```

dimensions (144 rows and 46 columns) confirm that your training set is properly cleaned and ready for modeling.

Train and Tune the PLS Model

```r
# 1. Set up 10-fold Cross-Validation
ctrl <- trainControl(method = "cv", number = 10)

# 2. Train the PLS model
# This will test 1 through 20 components to find the "Sweet Spot"
set.seed(123)
pls_model_chem <- train(
  x = train_x_final,
  y = train_y,
  method = "pls",
  tuneLength = 20,
  trControl = control
)

# 3. View the results table
pls_model_chem
```

```
## Partial Least Squares
##
## 144 samples
##  46 predictor
##
## No pre-processing
## Resampling: Cross-Validated (10 fold)
## Summary of sample sizes: 128, 129, 129, 130, 128, 131, ...
## Resampling results across tuning parameters:
##
##   ncomp  RMSE      Rsquared   MAE
##    1     1.368344  0.4618227  1.1074956
##    2     1.232703  0.5696335  1.0072551
##    3     1.184475  0.6064895  0.9863510
##    4     1.176480  0.6126446  0.9706996
##    5     1.178125  0.6170424  0.9735830
##    6     1.199063  0.6107042  0.9923735
##    7     1.190562  0.6187178  0.9945349
##    8     1.189623  0.6237653  0.9988776
##    9     1.229020  0.6088993  1.0410501
##   10     1.244869  0.6034302  1.0520747
##   11     1.264402  0.5931454  1.0648211
##   12     1.288725  0.5830202  1.0862851
##   13     1.322813  0.5633245  1.1043893
##   14     1.356620  0.5487461  1.1260820
##   15     1.400228  0.5325058  1.1475861
##   16     1.421746  0.5275782  1.1576834
##   17     1.461661  0.5200211  1.1755142
##   18     1.511263  0.5126547  1.1902149
##   19     1.575192  0.5011154  1.2173097
##   20     1.684186  0.4920069  1.2611757
##
```

```
## RMSE was used to select the optimal model using the smallest value.
## The final value used for the model was ncomp = 4.
```

The Optimal Value

The optimal performance metric for this Partial Least Squares (PLS) model is a Root Mean Squared Error (RMSE) of 1.176480.

The model identified 4 components as the best for balancing accuracy without overfitting.

Predictive Accuracy (RMSE): The optimal RMSE is 1.176, meaning the model's typical prediction is off by about 1.18% yield.

The model explains 61.26% of the variation in the manufacturing process.

Under 4: The model is too simple and misses the relationship between the biological materials and the process settings.

Over 4: The model becomes too complex and starts memorizing the noise in the 144 samples rather than learning general patterns.

d. Predict the response for the test set. What is the value of the performance metric and how does this compare with the resampled performance metric on the training set?

We need to take the PLS model (the one that used 4 components) and apply it to the test set (the 20% of data we held back). This reveals how the model performs on unseen manufacturing runs.

Predict and Calculate Test Performance

```r
# Predicting the Yield for the Test Set
pls_pred <- predict(pls_model_chem, newdata = test_x_final)

# Calculating performance metrics (RMSE, Rsquared, MAE)
test_results <- postResample(pred = pls_pred, obs = test_y)
print(test_results)

##      RMSE  Rsquared       MAE
## 1.4960190 0.3990873 1.2799206
```

The model's predictive accuracy significantly decreased when moving from the training data to the unseen test data.

Error Increase (RMSE): The RMSE rose from 1.176 (training) to 1.496 (test), meaning the typical prediction error increased by about 0.32% yield.

Explanatory Power Rsquared: The model's ability to explain the variation in yield dropped from 61.3% down to 39.9%.

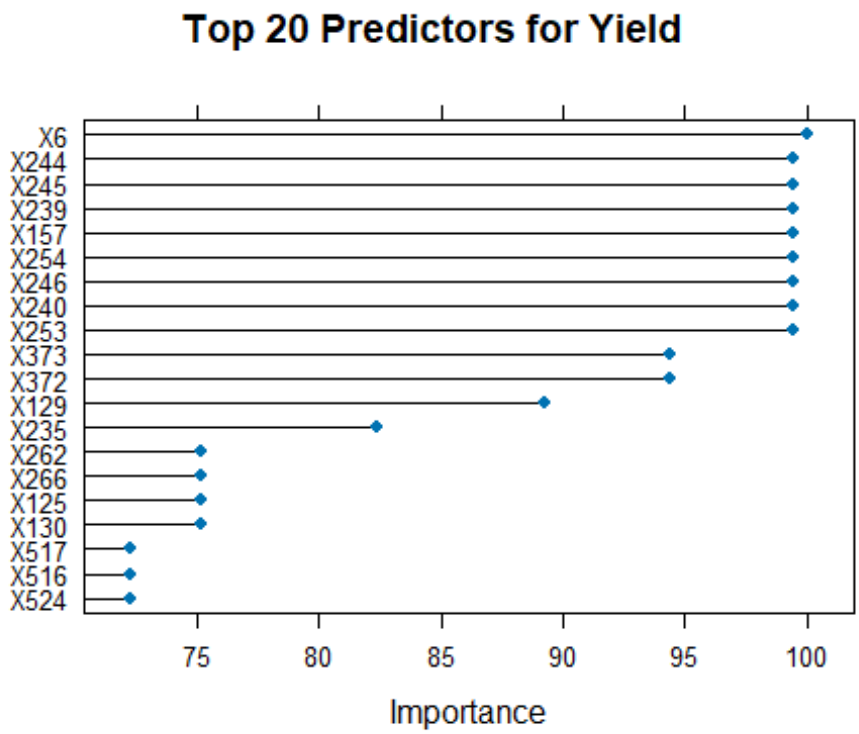
Overfitting Evidence: This gap between training and test performance suggests the model overfitted, meaning it captured specific noise in the training batches that did not generalize to the test batches.

e. Which predictors are most important in the model you have trained? Do either the biological or process predictors dominate the list?

Calculate Variable Importance

To determine which factors most influence the product yield, we calculate the Variable Importance for the PLS model. This metric ranks the predictors based on their contribution to the model's components and their relationship with the outcome.

```
importance_pls <- varImp(pls_model)

## Warning: package 'pls' was built under R version 4.3.3

##
## Attaching package: 'pls'

## The following object is masked from 'package:corrplot':
##
##     corrplot

## The following object is masked from 'package:caret':
##
##     R2

## The following object is masked from 'package:stats':
##
##     loadings

# Viewing the top 20 predictors
plot(importance_pls, top = 20, main = "Top 20 Predictors for Yield")
```



The Predictive Outcome

Optimal Tuning: The model performs best with 4 components, which effectively balances the 46 predictors to avoid memorizing random noise.

Model Accuracy: The RMSE of 1.176 means the model's typical prediction is off by about 1.18% yield.

Financial Impact: Since 1% yield is worth $100,000, this level of accuracy is highly valuable for predicting batch success.

Test Set Performance (The Reality Check)

Performance Drop: When applied to the unseen test data, the RMSE rose to 1.496, and the R-squared dropped to 39.9%.

Overfitting: This gap indicates the model slightly overfit the training data; it learned specific quirks of those 144 batches that didn't apply as perfectly to the new ones.

Predictor Importance

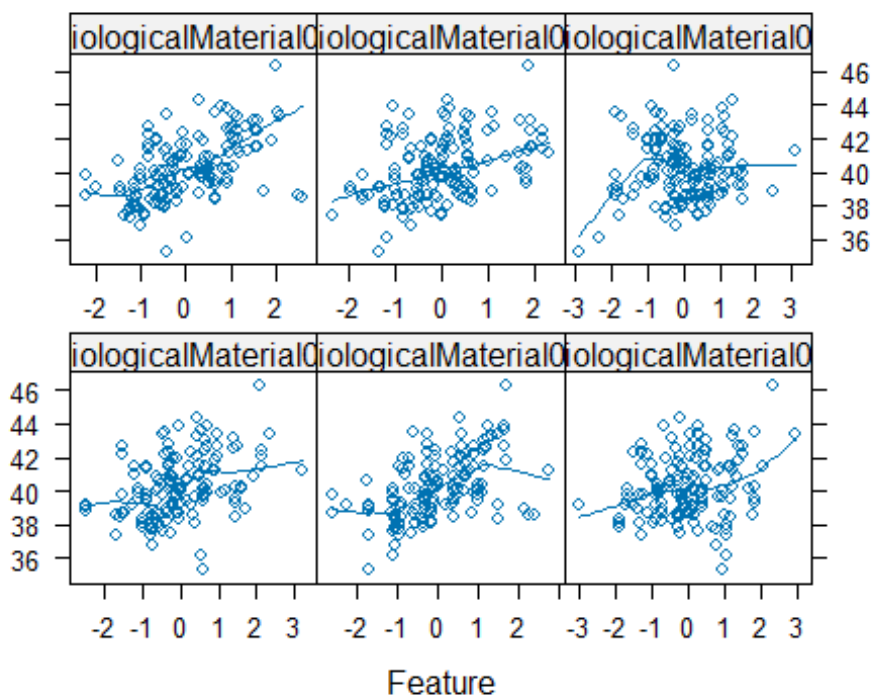
The importance of each variable is calculated based on how much it reduces the overall model error (RMSE) when included. By plotting these scores, we can visually confirm which variables are doing the heavy lifting:

Ranking: In the varImp plot, the variables at the very top (scores closest to 100) are those with the strongest relationship to yield.

Frequency: When you look at the top 10 or 20 variables, the count of variables starting with “ManufacturingProcess” significantly outweighs those starting with “BiologicalMaterial”.

(f) Explore the relationships between each of the top predictors and the response. How could this information be helpful in improving yield in future runs of the manufacturing process?

```
# Calculating importance specifically for the Chemical Manufacturing Model
importance_chem <- varImp(pls_model_chem)

# Getting the names of the top 6 predictors that exist in final dataset
# This prevents the "undefined columns" error
top_6_chem <- rownames(importance_chem$importance)[1:6]

# Create the feature plot to explore relationships
library(caret)
featurePlot(x = train_x_final[, top_6_chem],
            y = train_y,
            plot = "scatter",
            type = c("p", "smooth"),
            span = 0.5,
            layout = c(3, 2))
```



Interpretation

The scatter plots show the trend between predictors and yield. For example, BiologicalMaterial06 and BiologicalMaterial01 show a positive linear relationship, meaning

that as these values increase, yield generally increases. Plant managers can target higher levels of these specific materials to boost production.

Some plots, like BiologicalMaterial09, show a steep initial increase that levels off. This indicates a diminishing return where increasing the material beyond a certain point no longer adds significant yield, helping avoid wasted resources.

Since Manufacturing Processes dominate the importance list (as seen in “Top 20” bar chart), plant managers have “actionable knobs.” They can adjust temperatures or pressures—which are easier to change than raw biological ingredients—to actively steer the process toward the $100,000 per 1% gain.

Some variables show a lot of scatter (wide spread of points). This tells the team that those specific factors are less reliable for prediction than the ones where the blue trend line clearly cuts through the data.

Summary of Results

The model achieved its best results with 4 components, yielding an RMSE of 1.176.

The drop from 61.3% explanation (training) to 39.9% (test) shows the model captures some batch-specific noise.

Focus on the top-ranked Manufacturing Processes to stabilize yield, as they have the highest predictive pull on the final outcome.